

Quiz

Teoria Quiz Ing del software

Il team di sviluppo di un'azienda consiste di un senior software engineer e due sviluppatori junior. Usando un approccio agile, ogni iterazione impegna tutti e tre i membri del team per un mese ed occorrono tre iterazioni per completare lo sviluppo. Si assuma che non ci siano "change requests" e che il membro senior costi A Eur/mese ed i membri junior B Eur/mese. Qual'è il costo dello sviluppo usando un approccio agile?

Risposta: $3 \times (A + 2B)$

Il rischio R può essere calcolato come $R = P \cdot C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b \cdot S)$, dove b è una opportuna costante nota da dati storici aziendali.

- $b = 0.000061$
- $C = 184615.000000$
- $R = 533.000000$

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

- $b = 0.000061$
- $C = 184615.000000$
- $R = 533.000000$

Domanda 1 Scegli un'alternativa: 1.

Costo: 961582.

Costo: 621583.

Costo: 54364

Si consideri il seguente frammento di pseudo codice.

```
// Begin input/output connections for system components
```

```
Input 13 of SC3 = Output 13 of SC1
```

```
Input 14 of SC4 = Output 14 of SC1
```

```
Input 23 of SC3 = Output 23 of SC2
```

```
Input 24 of SC4 = Output 24 of SC2
```

```
Input 32 of SC2 = Output 32 of SC3
```

```
Input 34 of SC4 = Output 34 of SC3
```

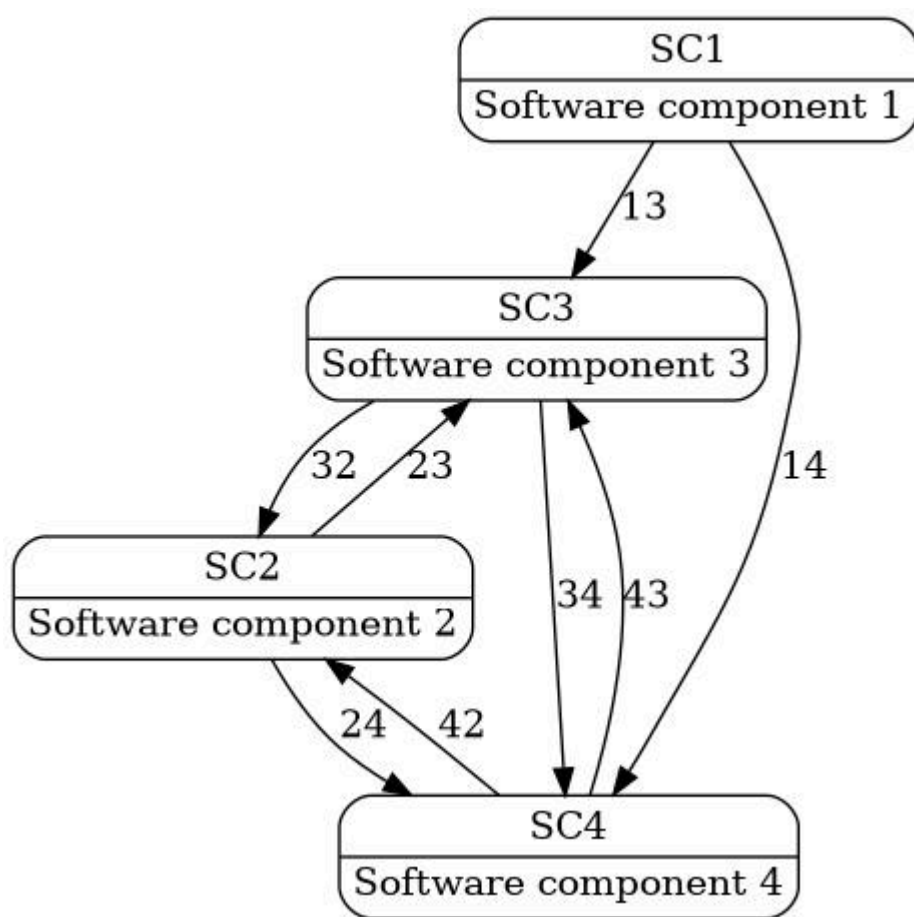
```
Input 42 of SC2 = Output 42 of SC4
```

```
Input 43 of SC3 = Output 43 of SC4
```

```
// End input/output connections for system components
```

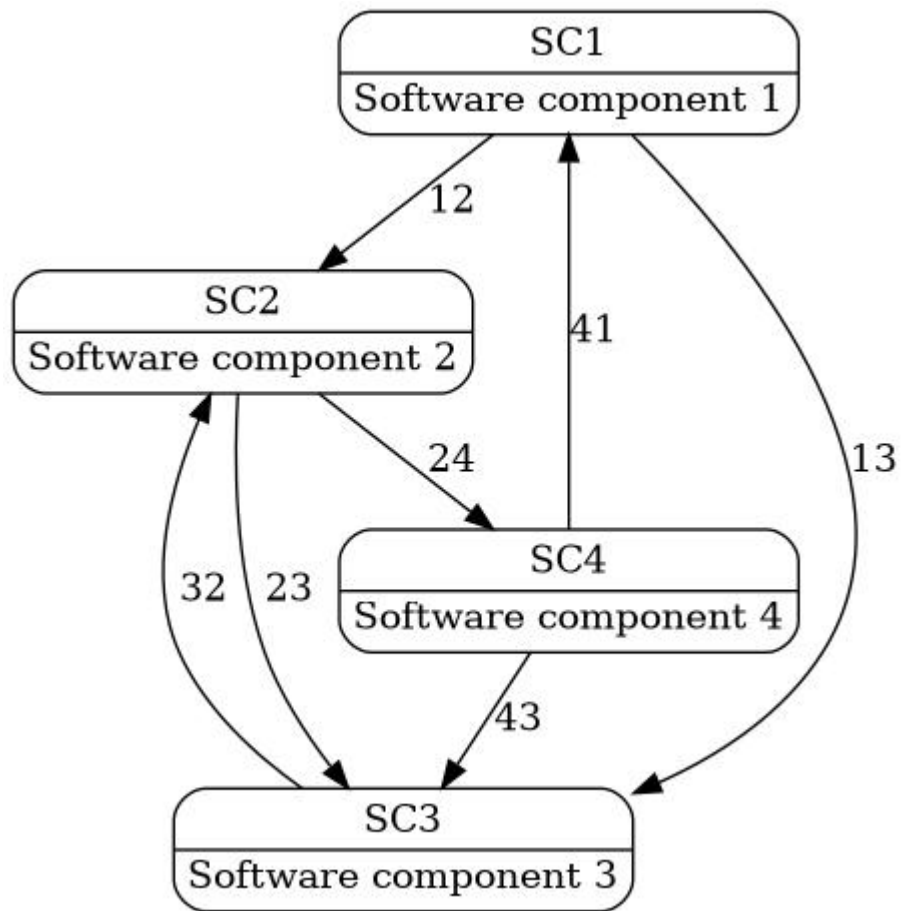
Quale delle seguenti architetture software meglio lo rappresenta ?

Domanda 2Scegli un'alternativa:

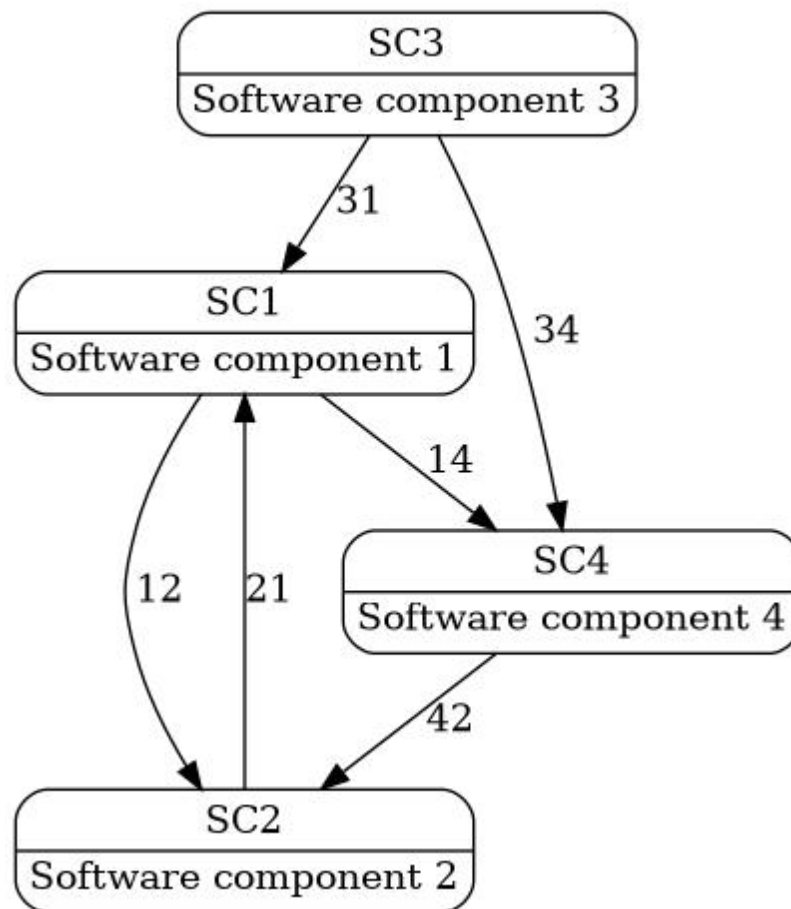


1.

2.



1.



Quale dei seguenti è un requisito non funzionale?

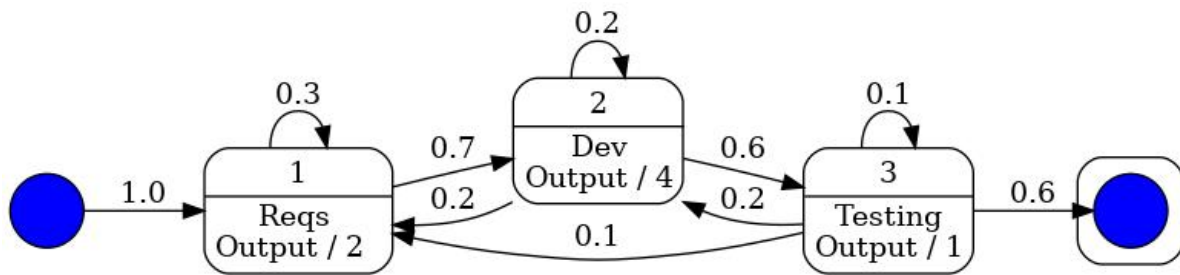
Domanda 3

1. L'output del sistema è sempre una matrice invertibile
2. L'output del sistema è sempre un vettore ordinato
3. L'utilizzo della RAM è sempre inferiore ai 4GBytes.

Un processo software di n stati può essere modellato con una coppia (A, Y) dove:

- **A è una matrice che definisce una catena di Markov con n stati etichettati con interi da 0 ad $n-1$. Lo stato iniziale è etichettato con 0 mentre lo stato finale con $n-1$.**
- **Y è un vettore di n elementi che definisce l'output della catena di Markov associando un output (se definito) a ciascuno stato.**

Quale delle seguenti rappresentazioni (A, Y) meglio modella il processo software in figura ?



Scegli un'alternativa:

1.

$A[0][1] = 1.0$

$A[1][1] = 0.3$

$A[1][2] = 0.7$

$A[2][1] = 0.2$

$A[2][2] = 0.2$

$A[2][3] = 0.6$

$A[3][1] = 0.1$

$A[3][2] = 0.2$

$A[3][3] = 0.1$

$A[3][4] = 0.6$

$A[4][4] = 1.0$

$Y[0] = -$

$Y[1] = 2$

$Y[2] = 2$

$Y[3] = 1$

$Y[4] = -$

2.

$A[0][1] = 1.0$

$A[1][1] = 0.4$

$A[1][2] = 0.6$

$A[2][1] = 0.1$

$A[2][2] = 0.1$

$$A[2][3] = 0.8$$

$$A[3][1] = 0.1$$

$$A[3][2] = 0.2$$

$$A[3][3] = 0.1$$

$$A[3][4] = 0.6$$

$$A[4][4] = 1.0$$

$$Y[0] = -$$

$$Y[1] = 2$$

$$Y[2] = 4$$

$$Y[3] = 1$$

$$Y[4] = -$$

3.

$$A[0][1] = 1.0$$

$$A[1][1] = 0.3$$

$$A[1][2] = 0.7$$

$$A[2][1] = 0.2$$

$$A[2][2] = 0.2$$

$$A[2][3] = 0.6$$

$$A[3][1] = 0.1$$

$$A[3][2] = 0.2$$

$$A[3][3] = 0.1$$

$$A[3][4] = 0.6$$

$$A[4][4] = 1.0$$

$$Y[0] = -$$

$$Y[1] = 2$$

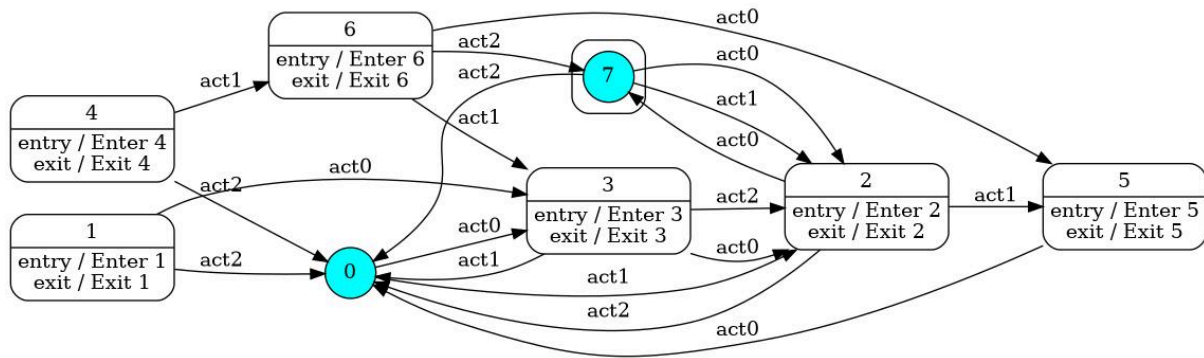
$$Y[2] = 4$$

$$Y[3] = 1$$

$$Y[4] = -$$

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act0 act0 act2 act0 act0

Test case 2: act1 act2 act1 act0

Test case 3: act1 act0

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Domanda 5

State coverage: 0.728

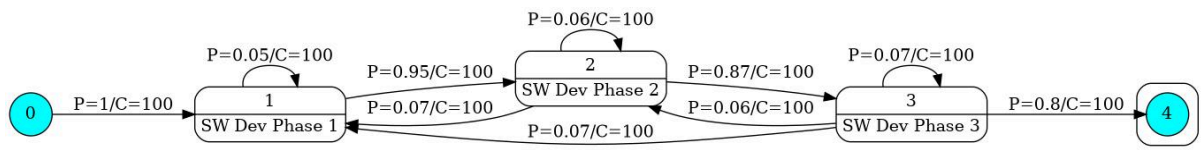
State coverage: 0.941

State coverage: 0.500

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Domanda 6

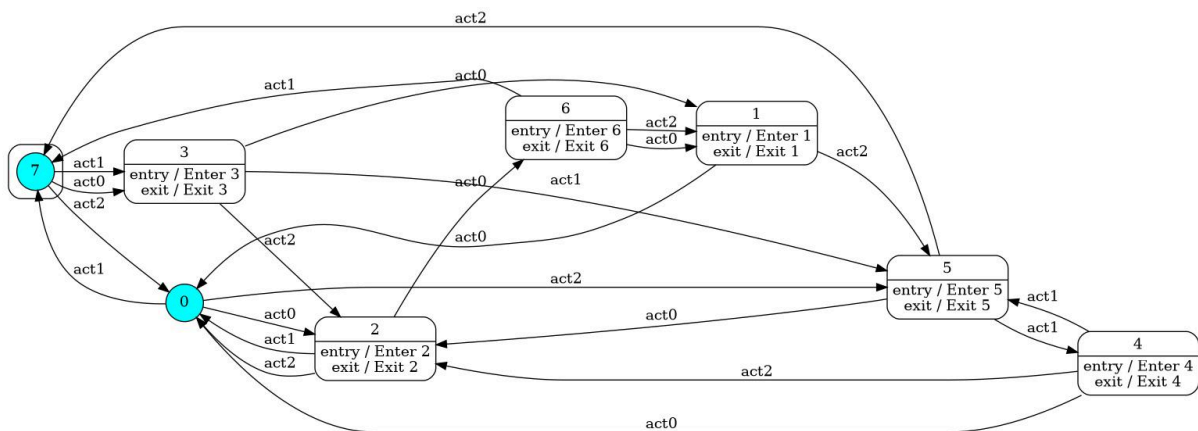
0.924539

0.816926

0.6612

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act1

Test case 2: act2 act0 act1 act2 act1

Test case 3: act0 act2 act2 act2

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Domanda 7

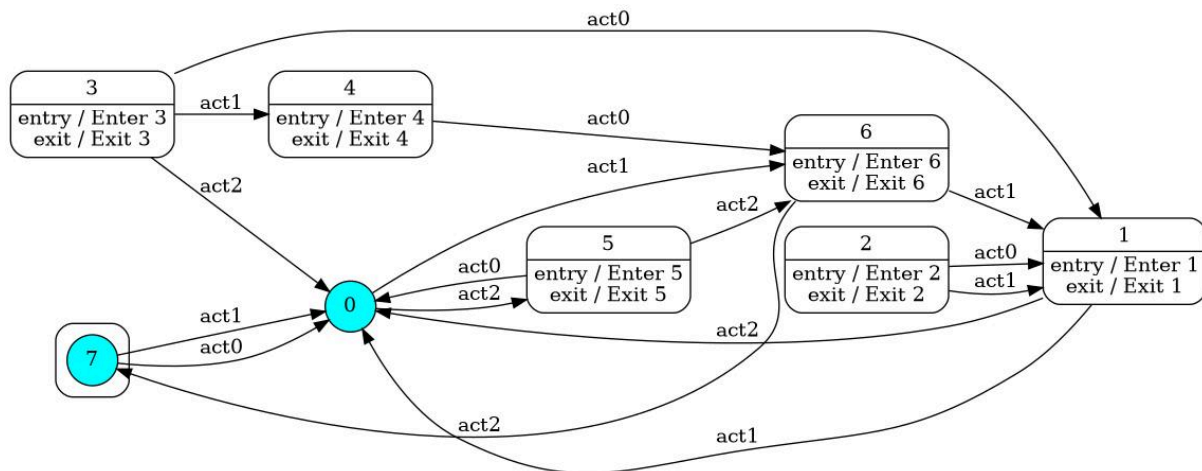
State coverage: 0.625

State coverage: 0.349

State coverage: 0.172

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act2 act2 act1 act2 act2

Test case 2: act1 act2

Test case 3: act2 act0 act1 act2

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

Transition coverage: 0.120

Transition coverage: 0.251

Transition coverage: 0.438

Si consideri la seguente funzione di transizione.

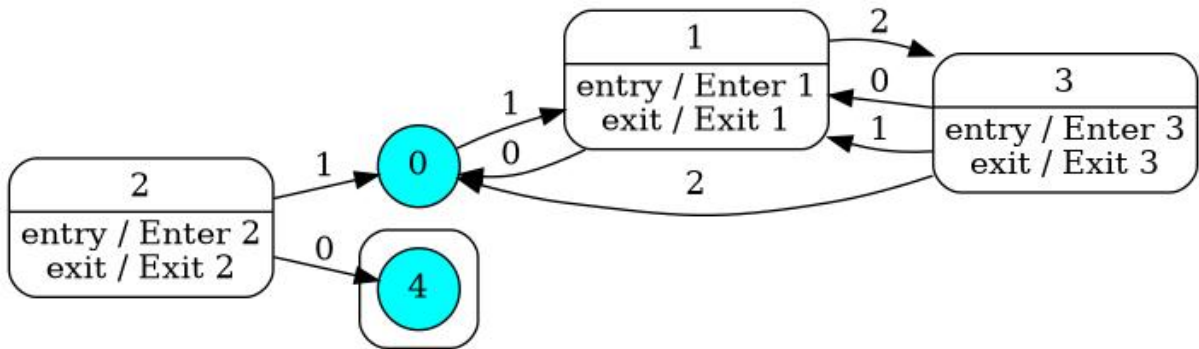
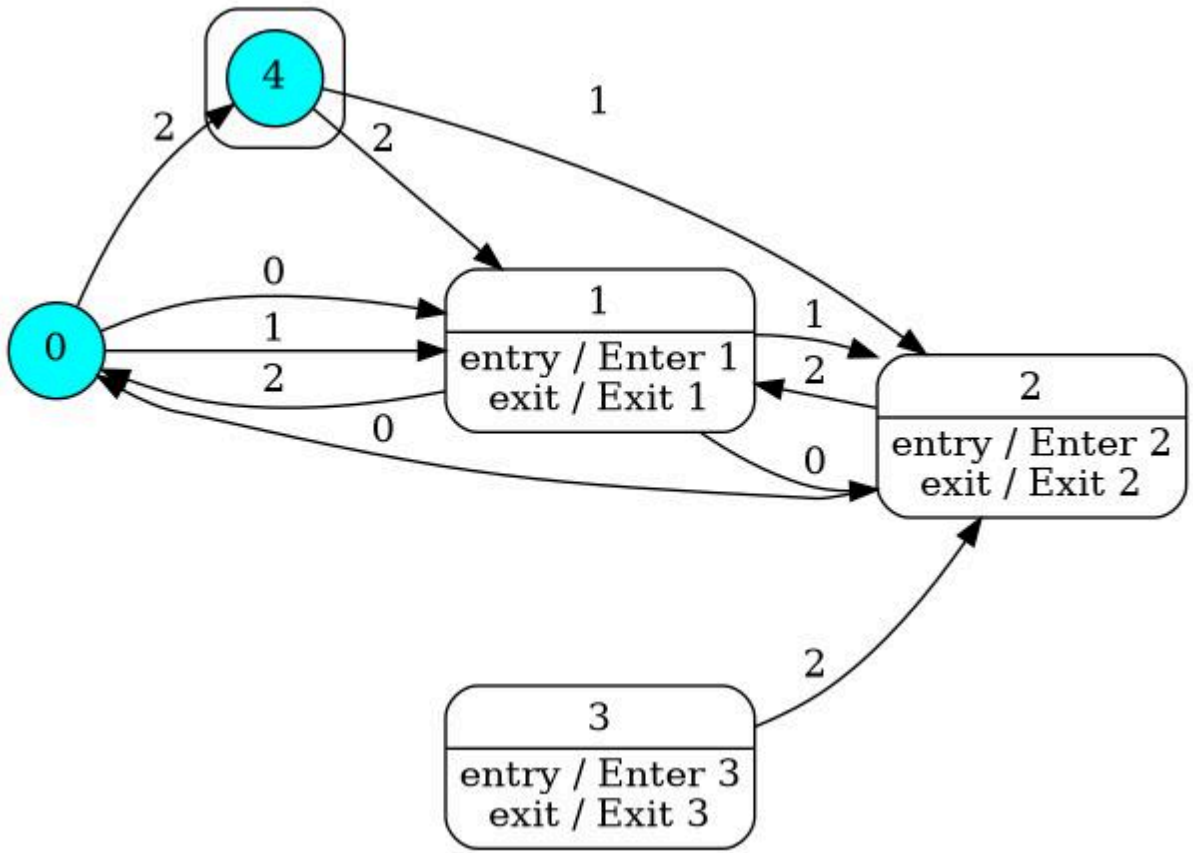
```

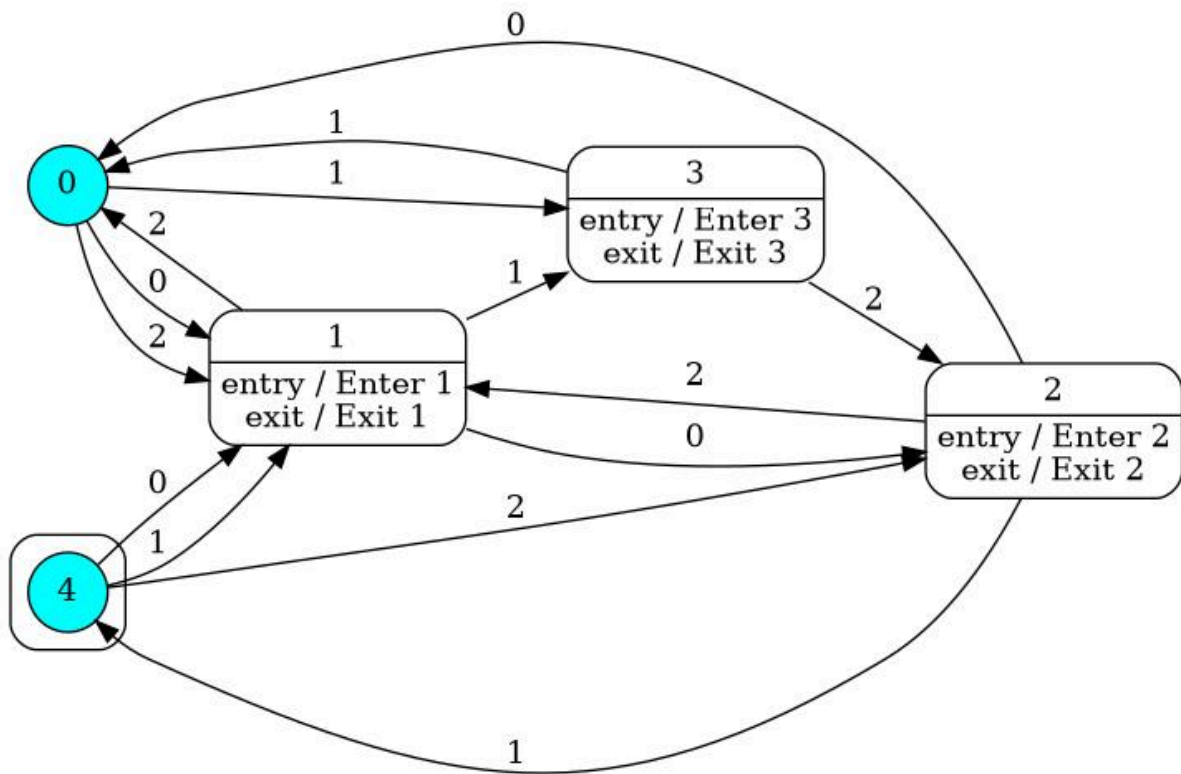
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (0);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (1);
        case 2: return (3);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (0);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    default: return (0);
}
} // f()

```

Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?





Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```

int f(int x[3])
{
    if ((2)*x[0] + (3)*x[1] + (-3)*x[2] <= 3)
        if ((2)*x[0] + (0)*x[1] + (-4)*x[2] <= 4)
            {return (0); }
        else
            {return (1); }
    else if ((3)*x[0] + (2)*x[1] + (-4)*x[2] <= -3)
        if ((-4)*x[0] + (3)*x[1] + (1)*x[2] <= 3)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */

```

Test 1: $x[0] = 5, x[1] = 1, x[2] = 3$.

Test 2: $x[0] = 1, x[1] = 4, x[2] = 4$.

Scegli un'alternativa:

Coverage: 0.400

Coverage: 0.817

Coverage: 0.683

La variabile x rappresenta lo stato presente del sistema, la variabile $x1$ rappresenta lo stato precedente ad x e la variabile $x2$ lo stato precedente ad $x1$.

Si consideri il seguente requisito:

La differenza tra il valore dello stato presente e di quello precedente è compresa tra -1 ed 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

`assert( ( x - x1) <= 1 )`

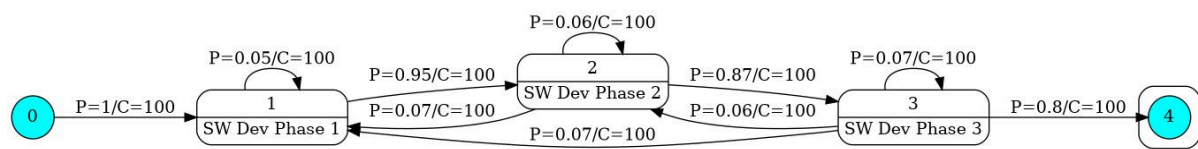
`assert( (-1 <= (x - x1)) && (( x1 - x) <= 1) )`

`assert( (-1 <= (x - x1)) && (( x - x1) <= 1) )`

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Domanda 12

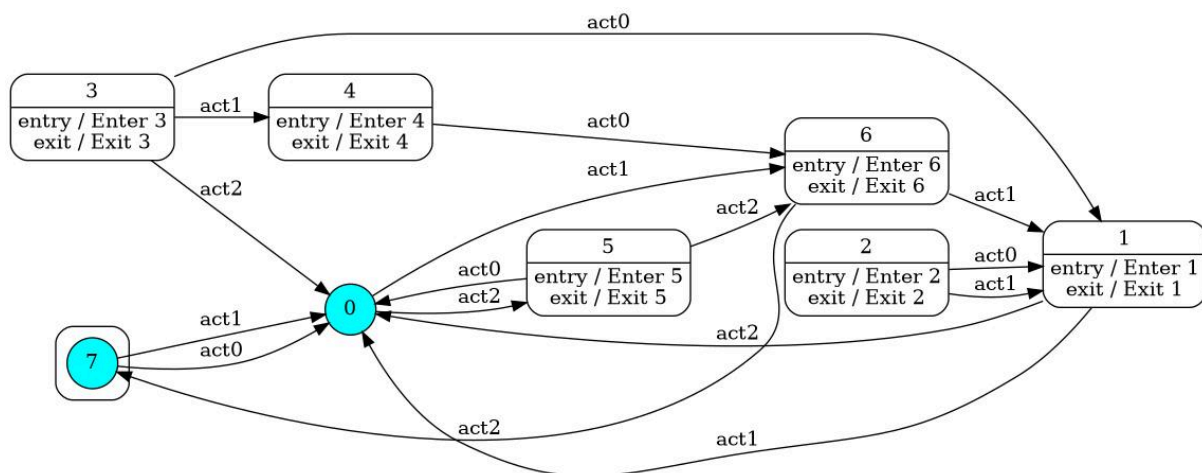
0.787594

0.0596922

0.0941

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act2 act2 act1 act2 act2

Test case 2: act1 act2

Test case 3: act2 act0 act1 act2

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Domanda 13

Transition coverage: 0.25

Transition coverage: 0.120

Transition coverage: 0.438

Quale delle seguenti frasi meglio descrive l'obiettivo del "validity check" che è parte della "requirements validation activity".

Domanda 14

Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del

"customer".

Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.

Focalizzandosi sui metodi agile di sviluppo del software, quale delle seguenti affermazioni è vera?

Domanda 15

Per evitare di sprecare tempo durante la fase di sviluppo del software, il customer non è mai coinvolto nel processo di sviluppo del software.

Le attività di definizione dei requisiti e di sviluppo sono interleaved.

Per evitare di sprecare tempo durante la fase di sviluppo del software, questa inizia solo quando i requisiti sono stati completamente definiti.

La variabile x rappresenta lo stato presente del sistema, la variabile x1 rappresenta lo stato precedente ad x e la variabile x2 lo stato precedente ad x1.

Si consideri il seguente requisito:

Se il sistema è nello stato 1 allora o permane nello stato di 1 oppure passa allo stato 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

Domanda 16

`assert((x==1) ? ((x1==1) || (x1==2)) : 1)`

`assert((x1==1) ? ((x==1) || (x==2)) : 1)`

`assert((x1 ==1) ? ((x==1) || (x==2)) : 0)`

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di completezza" che è parte della "requirements validation activity".

Scegli un'alternativa:

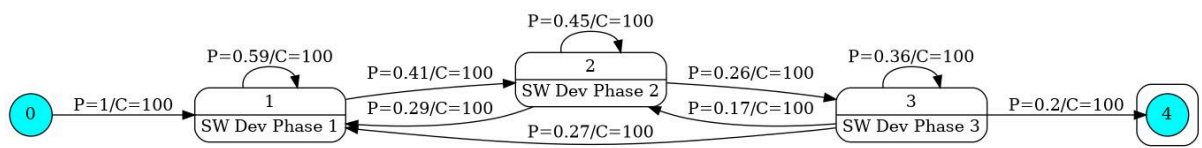
- 1. Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer.**
2. Assicurarsi che per ogni requisito sia stato implementato nel sistema.

3. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di transizioni necessario per lasciare la "SW Dev Phase 1"?

Domanda 18 Scegli un'alternativa:

0.693621

2.43902

1.52881

Si consideri un software sviluppato seguendo un approccio iterativo implementato con due fasi: F1 seguita da F2. Ciascuna fase ha costo 10000 Eur e deve essere ripetuta una seconda volta con probabilità 0.1. Qual'è il costo atteso dello sviluppo dell'intero software?

Scegli un'alternativa:

25000

30000

22000

Quale delle seguenti frasi meglio descrive il criterio di "requirements verifiability" che è parte della "requirements validation activity".

Domanda 20

Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test

che può dimostrare che il sistema sviluppato soddisfa il requisito considerato.

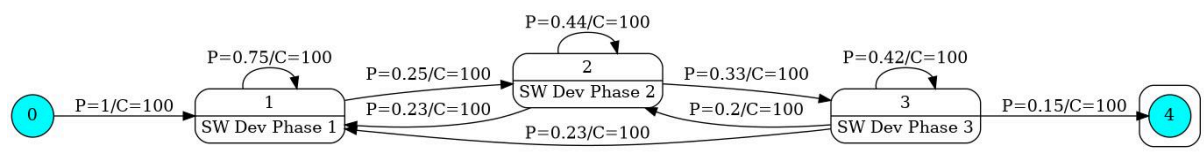
Per ciascuna componente del sistema, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che essa soddisfa tutti i requisiti.

Per ciascuna coppia di componenti, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che l'interazione tra le componenti soddisfa tutti i requisiti di interfaccia.

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di iterazioni della "SW Dev Phase 2"?

Domanda 21

0.785714

1.97891

5.0084

Il rischio R può essere calcolato come $R = P * C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b * S)$, dove b è una opportuna costante note da dati storici aziendali.

$b = 0.000052C = 1067178.000000R = 581.000000$

- $b = 0.000052$

- $C = 1067178.000000$
- $R = 581.000000$

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

Domanda 22

Costo: 145378

Costo: 132312

Costo: 141660

La WBS (Work Breakdown Structure) di un progetto consiste di 6 tasks: T0, T1, T2, T3, T4, T5.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
5	0
5	1
5	2
5	3
5	4

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	3
1	2
2	3
3	2
4	1
5	4

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling* .
 Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:
 i è il time slot (mese, nel caso in cui l'effort è misurato in PM); j è il task k è
 l'effort (positivo) dedicato al task j nel time slot i

- i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
- j è il task
- k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo.
 Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

1.

Time slot (Month)	Task	Effort (PM)
0	1	2
0	2	2
1	1	4

Time slot (Month)	Task	Effort (PM)
2	2	2
3	4	2
5	0	2
7	3	1
9	5	3

2.

Time slot (Month)	Task	Effort (PM)
0	1	2
0	2	2
1	1	4
2	2	2
3	4	2
5	0	2
7	3	1
9	5	3

3.

Time slot (Month)	Task	Effort (PM)
0	1	2
0	2	2
1	1	4
2	2	2
3	4	2
5	0	2
7	3	1
9	5	3

Si consideri un software sviluppato seguendo un approccio iterativo implementato con due fasi: F1 seguita da F2. Ciascuna fase ha costo 1000 Eur e deve essere ripetuta una seconda volta con probabilità 0.5. Qual'è il costo atteso dello sviluppo dell'intero software?

Domanda 24

3000

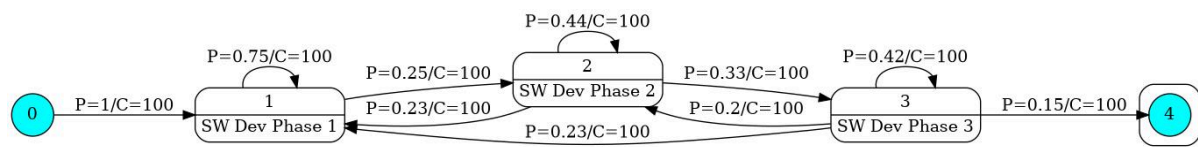
2000

5000

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di iterazioni della "SW Dev Phase 3"?

Domanda 25

0.724138

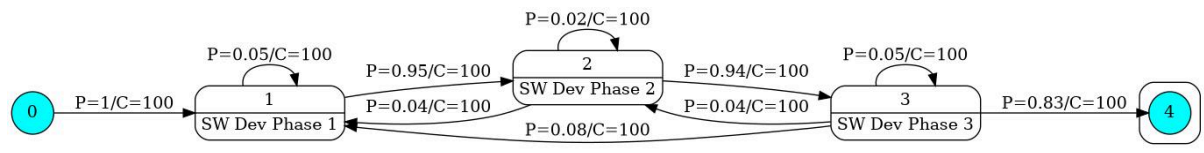
4.48153

2.88738

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Domanda 26

0.74119

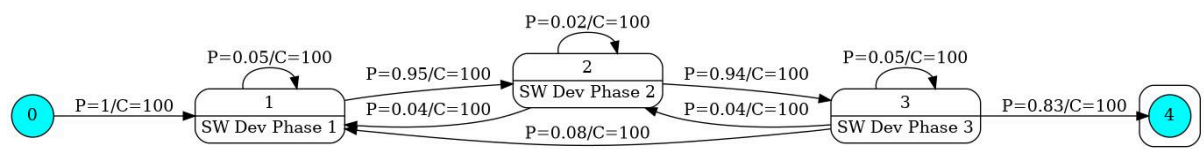
0.496489

0.222563

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

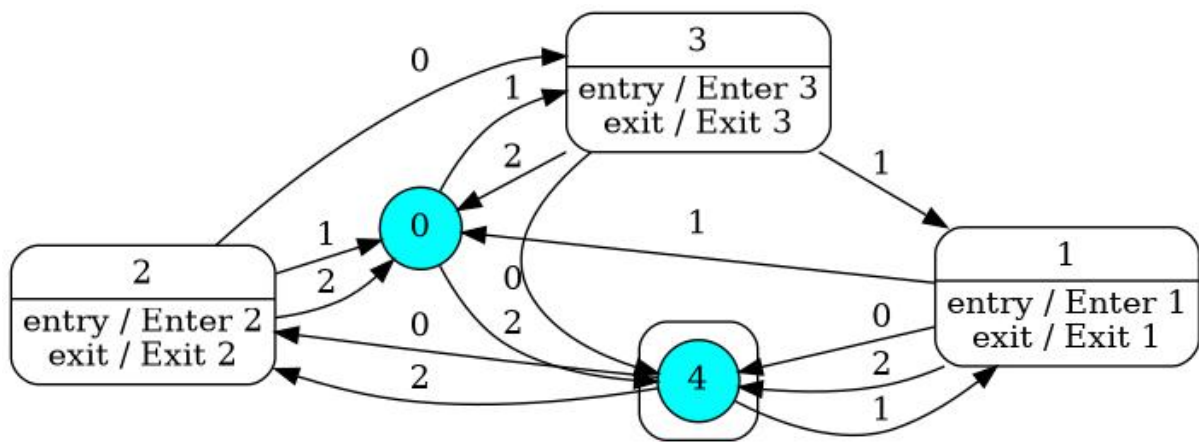
Domanda 27

0.850792

0.505184

0.0672

Si consideri lo state diagram in figura.



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (3);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (3);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (2);
        case 2: return (1);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    default: return (0);
}
} // f()

```

1.

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (2);
        case 2: return (2);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (2);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (2);
        case 1: return (4);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (2);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (2);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    default: return (0);
}
} // f()

```

2.

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (0);
        case 1: return (3);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (4);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (2);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    default: return (0);
}
} // f()

```

3.

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i test cases dati.

```

int f(int x[3])
{
    if ((1)*x[0] + (1)*x[1] + (-4)*x[2] <= -3)
        if ((-4)*x[0] + (2)*x[1] + (-3)*x[2] <= 0)
            {return (0); }
        else
            {return (1); }
    else if ((2)*x[0] + (-2)*x[1] + (-1)*x[2] <= -1)
        if ((3)*x[0] + (3)*x[1] + (4)*x[2] <= -4)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */

```

Test 1: $x[0] = -2, x[1] = -3, x[2] = -2$.

Test 2: $x[0] = 0, x[1] = -3, x[2] = 1$.

Test 3: $x[0] = 3, x[1] = 5, x[2] = 1$.

Test 4: $x[0] = -2, x[1] = 2, x[2] = 1$.

Domanda 29

Coverage: 0.970

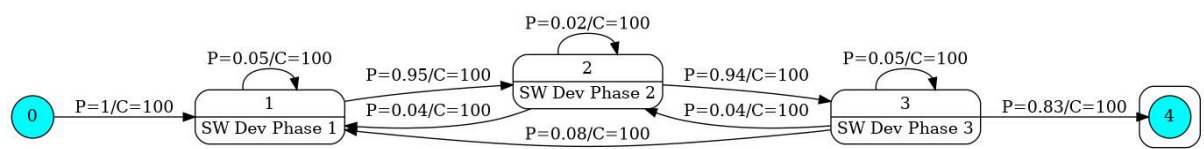
Coverage: 0.897

Coverage: 0.800

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Domanda 30

0.0601833

0.0902

0.724814

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Se lo stato vale 0 per due transizioni consecutive, allora alla transizione successiva lo stato avrà valore 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

`assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 0 )`

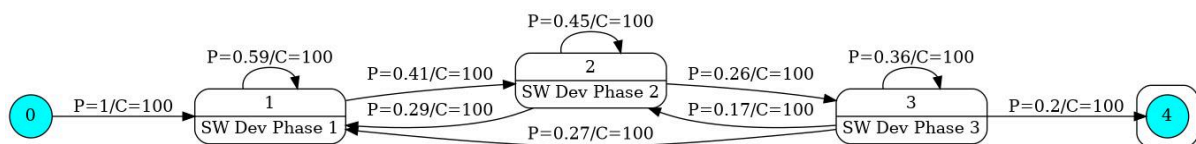
`assert( (x == 0) && (x1 == 0) ? (x2 == 1) : 1 )`

`assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 1 )`

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di transizioni necessario per lasciare la "SW Dev Phase 2"?

Domanda 32

1.81818

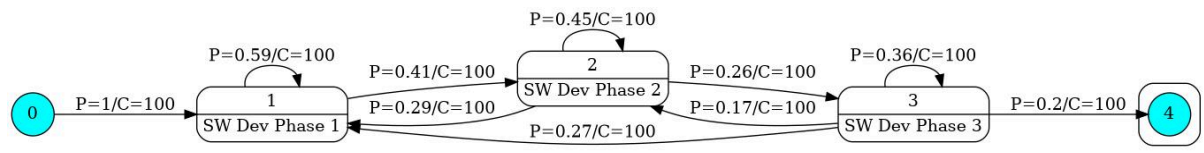
0.935294

0.577769

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di transizioni necessario per lasciare la "SW Dev Phase 3"?

Domanda 33

1.00118

0.322384

1.5625

Si consideri il seguente frammento di pseudo codice.

```
// Begin input/output connections for system components
```

```
Input 12 of SC2 = Output 12 of SC1
```

```
Input 14 of SC4 = Output 14 of SC1
```

```
Input 23 of SC3 = Output 23 of SC2
```

```
Input 24 of SC4 = Output 24 of SC2
```

```
Input 31 of SC1 = Output 31 of SC3
```

```
Input 32 of SC2 = Output 32 of SC3
```

```
Input 34 of SC4 = Output 34 of SC3
```

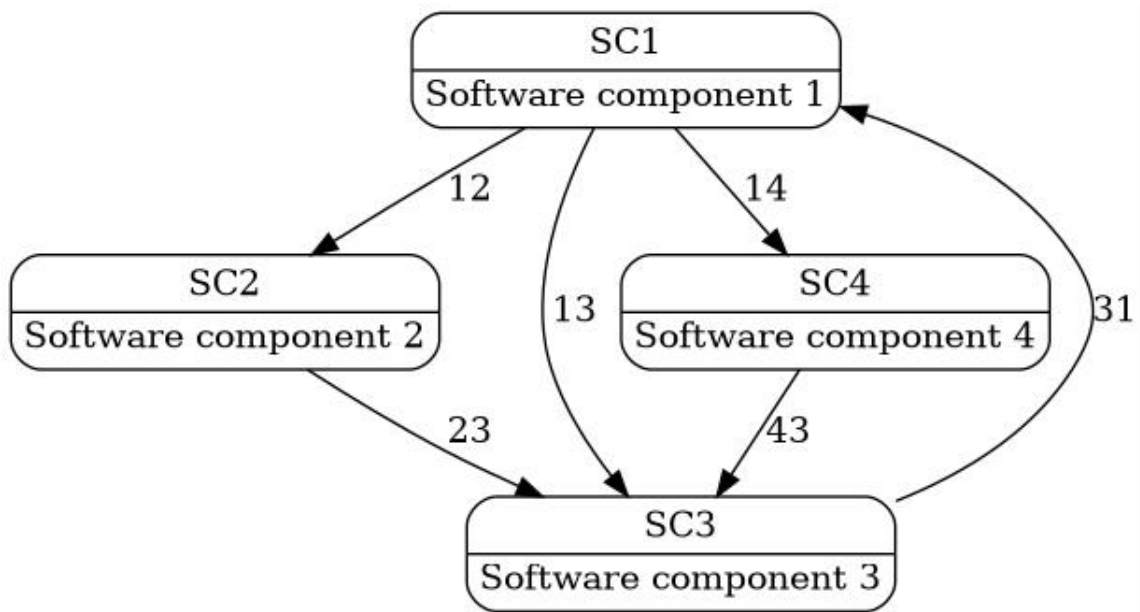
```
Input 41 of SC1 = Output 41 of SC4
```

```
Input 42 of SC2 = Output 42 of SC4
```

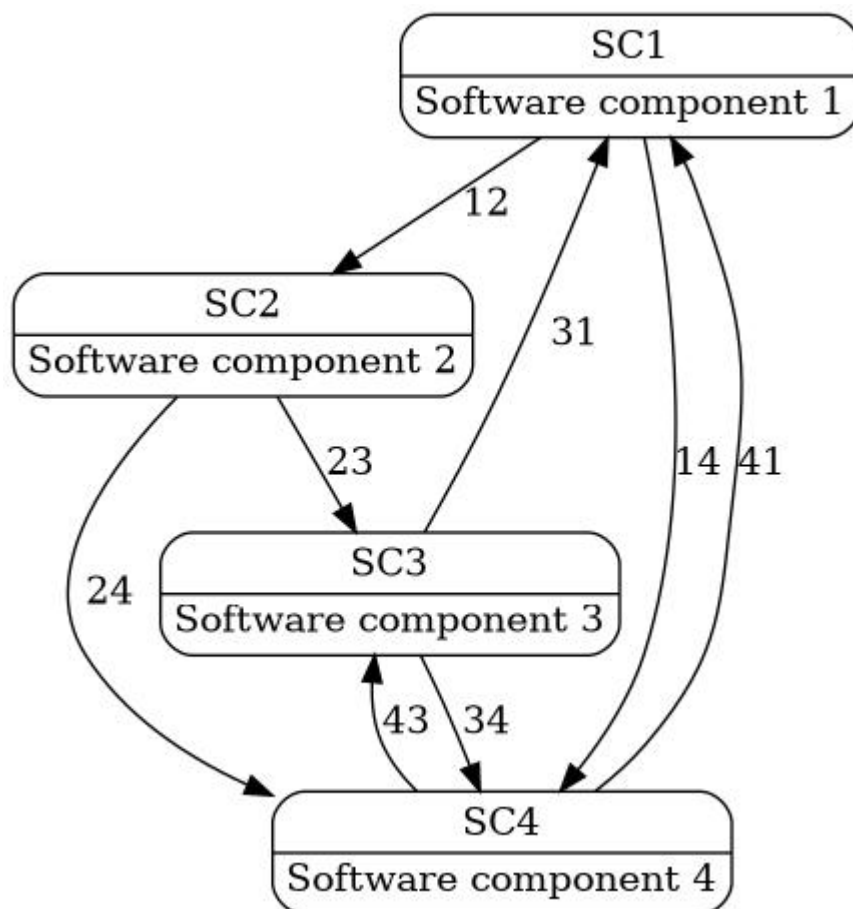
```
Input 43 of SC3 = Output 43 of SC4
```

```
// End input/output connections for system components
```

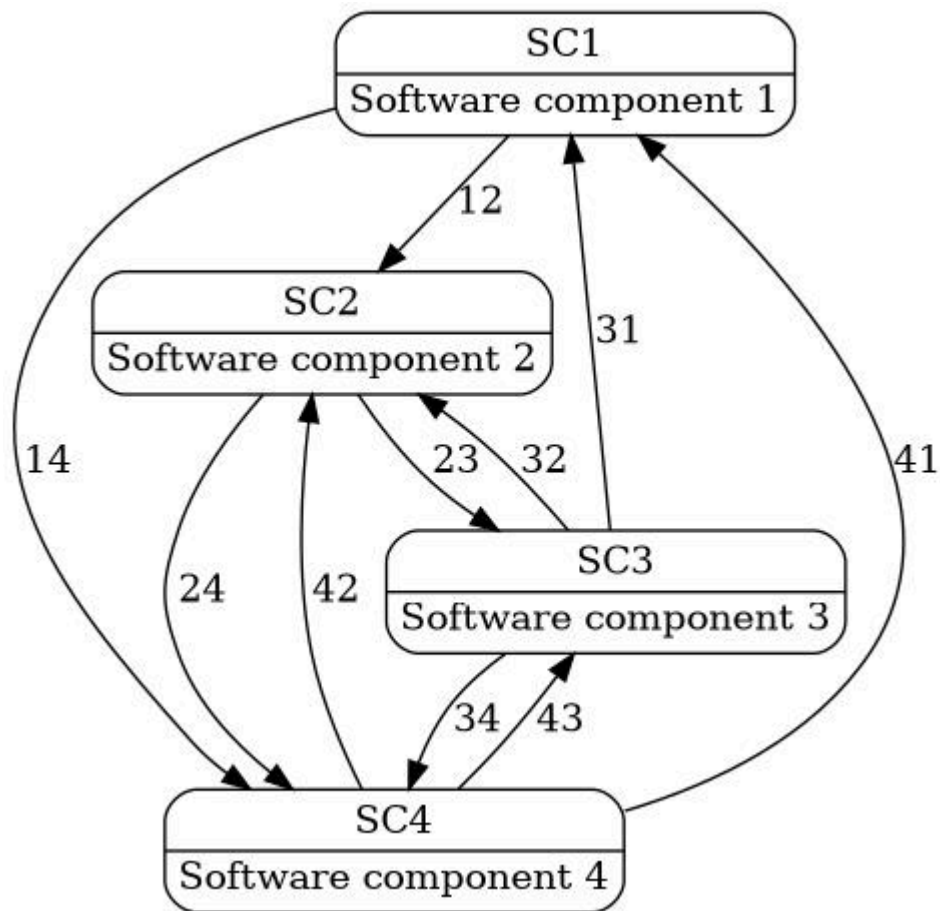
Quale delle seguenti architetture software meglio lo rappresenta ?



1. _____

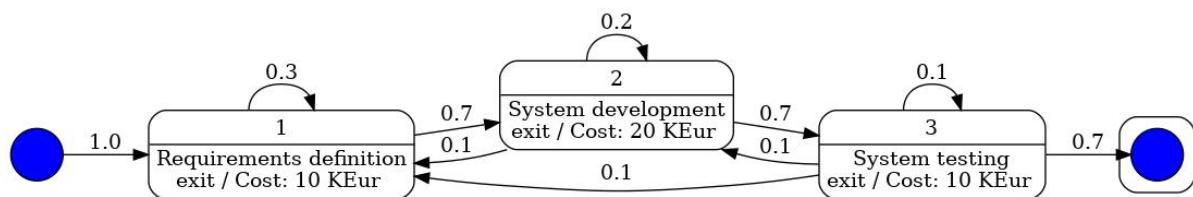


2. _____



3.

Si consideri il processo software software in figura



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 60K EUR ?

Scegli un'alternativa:

0.356

0.234

0.113

Un processo di sviluppo agile consiste di varie iterazioni. Alla fine di ogni iterazione vengono prese in considerazione le "change requests" e, se ve ne sono, l'iterazione viene ripetuta. Sia p la probabilità che ci siano "change

requests" all fine di una iterazione e sia A il costo di una iterazione. Il valore atteso del costo per l'iterazione è:

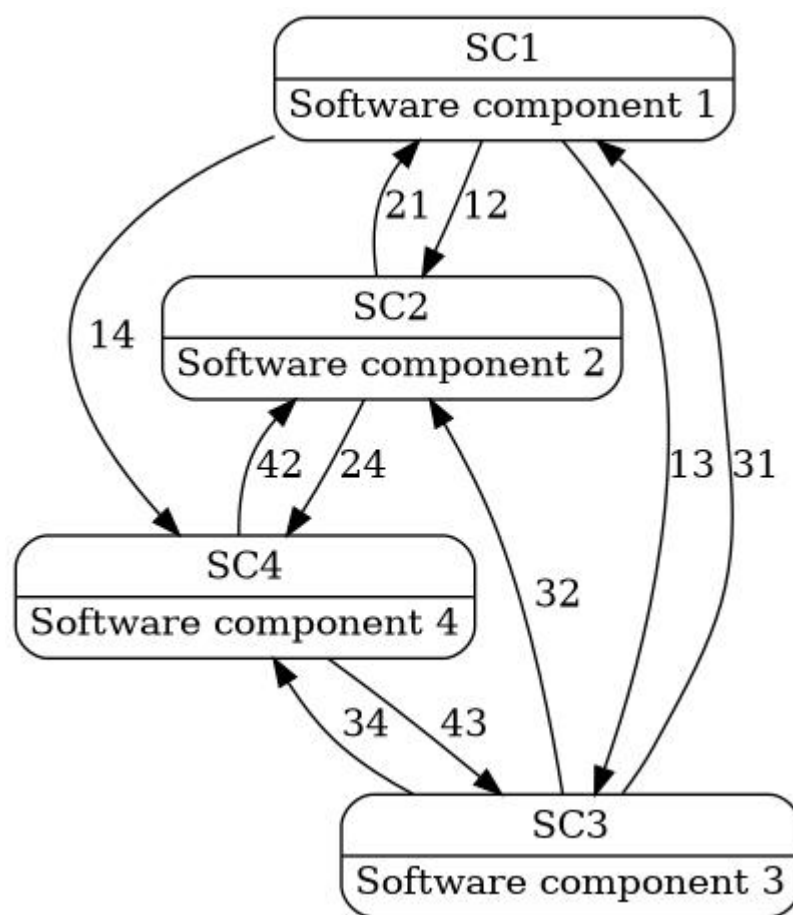
Domanda 36

$p \cdot A$

$(1 + p) \cdot A$

A

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Domanda 37

Scegli un'alternativa:

1. // Begin input/output connections for system components

Input 13 of SC3 = Output 13 of SC1

Input 14 of SC4 = Output 14 of SC1

Input 21 of SC1 = Output 21 of SC2

Input 23 of SC3 = Output 23 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
// End input/output connections for system components

2.

// Begin input/output connections for system components
Input 13 of SC3 = Output 13 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components

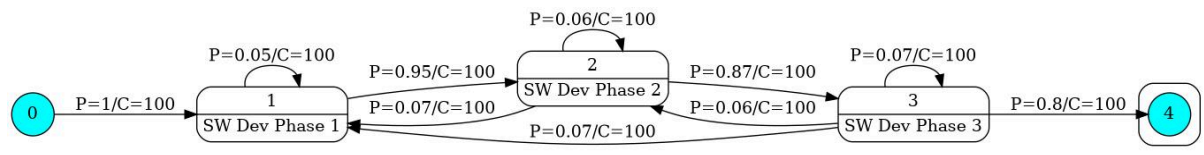
3.

// Begin input/output connections for system components
Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Domanda 38

0.408143

0.1202

0.686118

La WBS (Work Breakdown Structure) di un progetto consiste di 6 tasks: T0, T1, T2, T3, T4, T5.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
3	0
3	1
3	2
4	0
4	1
4	2
5	3
5	4

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	2
1	6
2	4
3	1

Task ID	Effort
4	2
5	3

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

i è il time slot (mese, nel caso in cui l'effort è misurato in PM); j è il task k è l'effort (positivo) dedicato al task j nel time slot i

- i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
- j è il task
- k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

1.

Time slot (Month)	Task	Effort (PM)
0	1	2
0	2	2
1	1	4
2	2	2
3	4	2
5	0	2
7	3	1
9	5	3

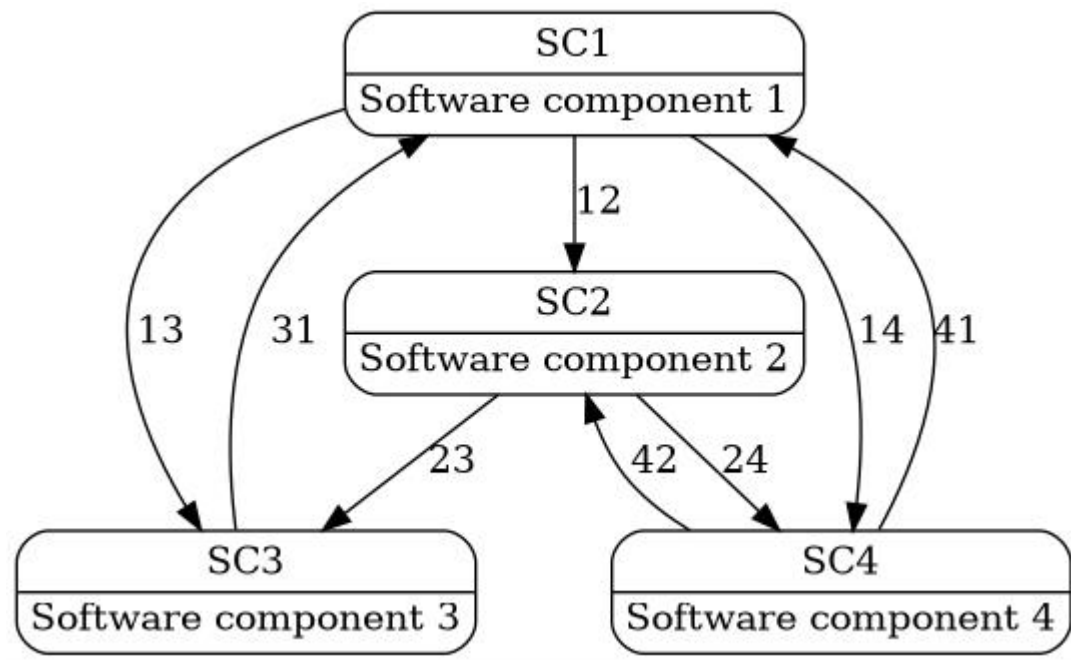
2.

Time slot (Month)	Task	Effort (PM)
0	2	4
1	1	4
2	0	2
2	1	2
3	4	2
4	3	1
5	5	3

3.

Time slot (Month)	Task	Effort (PM)
0	2	4
1	0	2
1	1	2
2	1	4
3	3	1
4	4	1
5	5	3

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Domanda 40

Scegli un'alternativa:

1. // Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1

Input 13 of SC3 = Output 13 of SC1

Input 14 of SC4 = Output 14 of SC1

Input 23 of SC3 = Output 23 of SC2

Input 24 of SC4 = Output 24 of SC2

Input 31 of SC1 = Output 31 of SC3

Input 41 of SC1 = Output 41 of SC4

Input 42 of SC2 = Output 42 of SC4

// End input/output connections for system components

2.

// Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1

Input 21 of SC1 = Output 21 of SC2

Input 24 of SC4 = Output 24 of SC2

Input 34 of SC4 = Output 34 of SC3

Input 41 of SC1 = Output 41 of SC4

Input 42 of SC2 = Output 42 of SC4

// End input/output connections for system components

3.

```
// Begin input/output connections for system components
```

Input 24 of SC4 = Output 24 of SC2

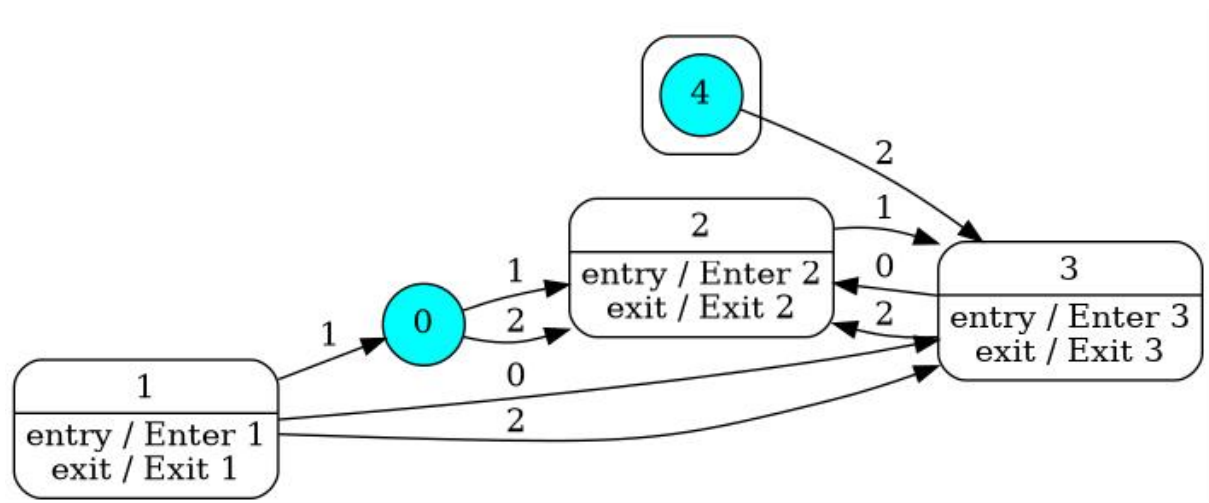
Input 32 of SC2 = Output 32 of SC3

Input 41 of SC1 = Output 41 of SC4

Input 43 of SC3 = Output 43 of SC4

```
// End input/output connections for system components
```

Si consideri lo state diagram in figura.



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa:

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (0);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (2);
        case 1: return (1);
        case 2: return (3);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    default: return (0);
}
} // f()

```

1.

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (0);
        case 1: return (2);
        case 2: return (2);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (0);
        case 2: return (3);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (2);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (2);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    default: return (0);
}
} // f()

```

2.

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (3);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (1);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (0);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    default: return (0);
}
} // f()

```

3.

Quale delle seguenti affermazioni è vera riguardo ai metodi agile ?

Domanda 42

I metodi agile sono metodi di sviluppo plan-driven.

I metodi agile sono metodi di sviluppo orientato al riuso.

I metodi agile sono metodi di sviluppo incrementale.

Si consideri un software sviluppato seguendo un approccio iterativo implementato con due fasi: F1 seguita da F2. Ciascuna fase ha costo A. Con probabilità p potrebbe essere necessario ripetere F1 una seconda volta. Con probabilità q potrebbe essere necessario ripetere F2 una seconda volta. Qual'è il costo atteso dello sviluppo dell'intero software?

Domanda 43

1. $A \cdot (3 + p + q)$

2. $A \cdot (1 + p + q)$

3. $A \cdot (2 + p + q)$

L'attività di *unit testing* di una funzione è stata suddivisa nei seguenti task:

Task identifier	Task description
TDEF	Definition of test cases
TORC	Definition of test oracle
TEXE	Esecuzione dei casi di test
TVAL	Valutazione dei casi di test

Quale dei seguenti insiemi di vincoli di precedenza (dove $A < B$ significa che A precede B) deve necessariamente valere, considerata la descrizione data dei tasks?

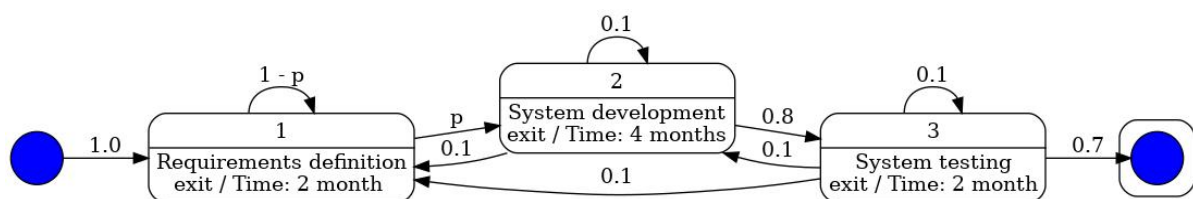
Scegli un'alternativa:

1. $TEXE < TORC$, $TVAL < TEXE$, $TVAL < TORC$.

2. $TEXE < TVAL$, $TDEF < TEXE$, $TORC < TVAL$.

3. $TVAL < TEXE$, $TORC < TDEF$, $TORC < TVAL$.

Si consideri il processo software in figura



Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini in esattamente 8 mesi sia almeno 0.5 ?

Scegli un'alternativa:

0.893

0.645

0.959

Un processo di sviluppo agile consiste di 3 iterazioni identiche di costo A. Alla fine di ogni iterazione vengono prese in considerazione le "change requests" e, se ve ne sono, l'iterazione viene ripetuta. Sia p la probabilità che ci siano "change requests" all fine di una iterazione. Il valore atteso del costo del progetto è:

46Scegli un'alternativa:

1. $3 \cdot (1 + p) \cdot A$

2. $3 \cdot (A + p)$

3. $3 \cdot p \cdot A$

La variabile x rappresenta lo stato presente del sistema, la variabile x1 rappresenta lo stato precedente ad x e la variabile x2 lo stato precedente ad x1.

Si consideri il seguente requisito:

Se lo stato vale 0 allora dopo al più 2 transizioni lo stato prende il valore 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

a. $\text{assert}((x == 0) ? (x1 == 1) \parallel (x2 == 1) : 1)$

b. $\text{assert}((x2 == 0) ? (x1 == 1) \parallel (x == 1) : 0)$

c. $\text{assert}((x2 == 0) ? (x1 == 1) \parallel (x == 1) : 1)$

Si consideri la seguente funzione di transizione.

```

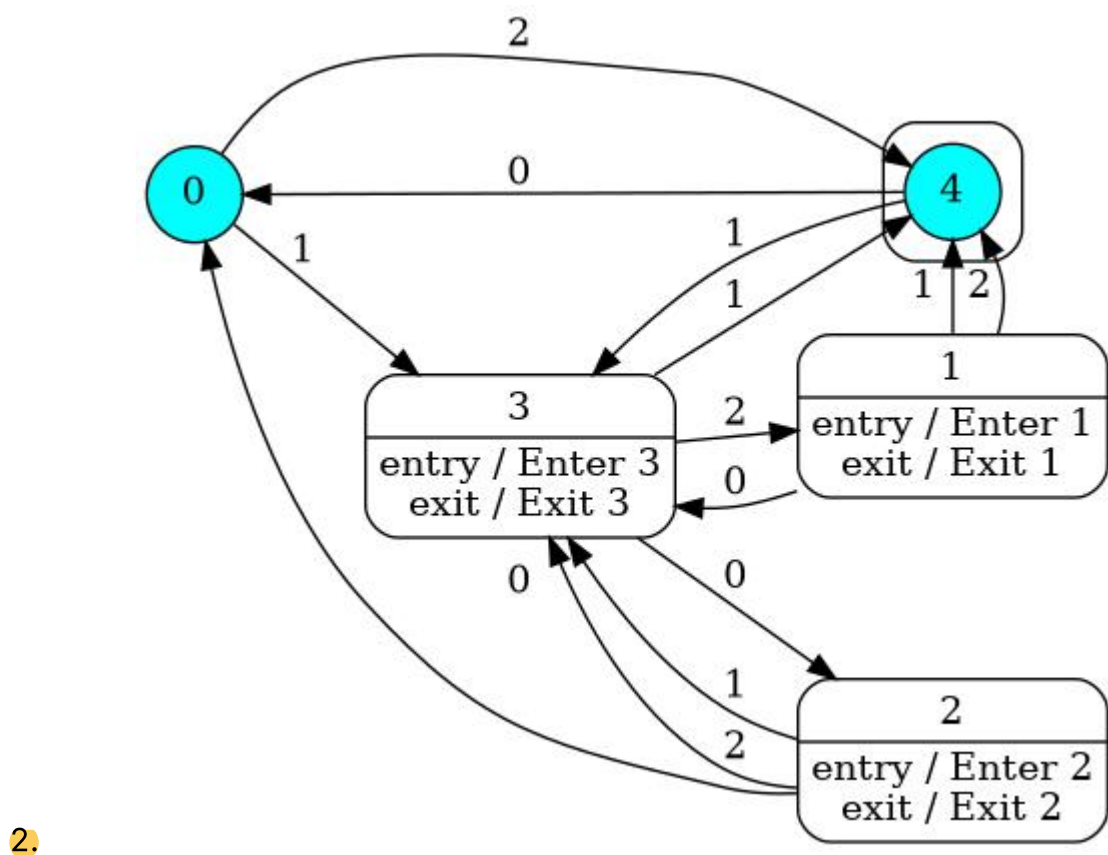
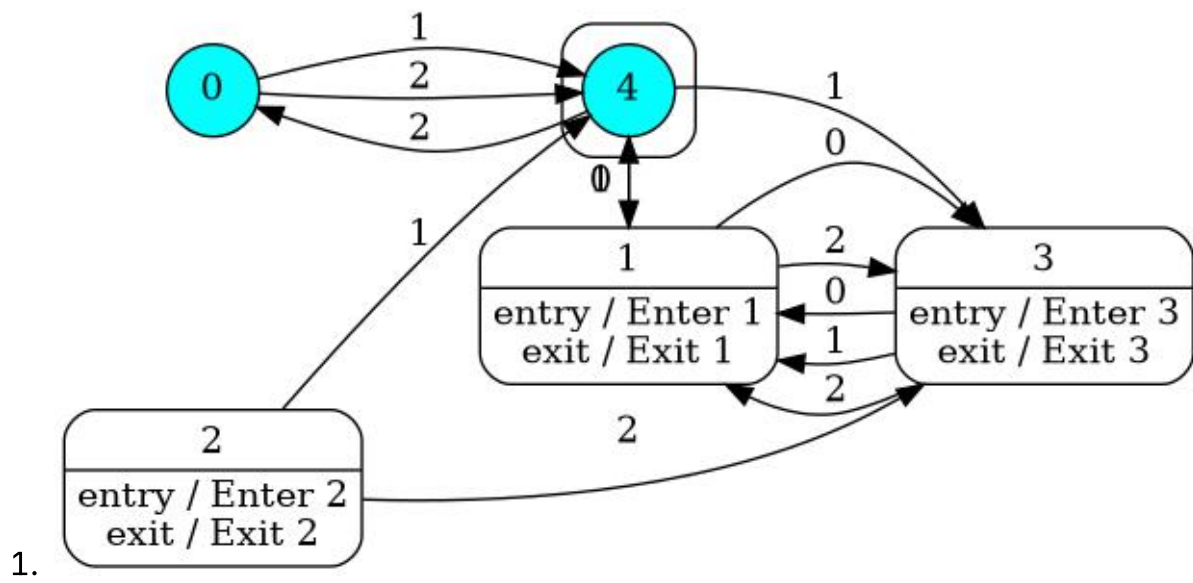
#include "main.h"

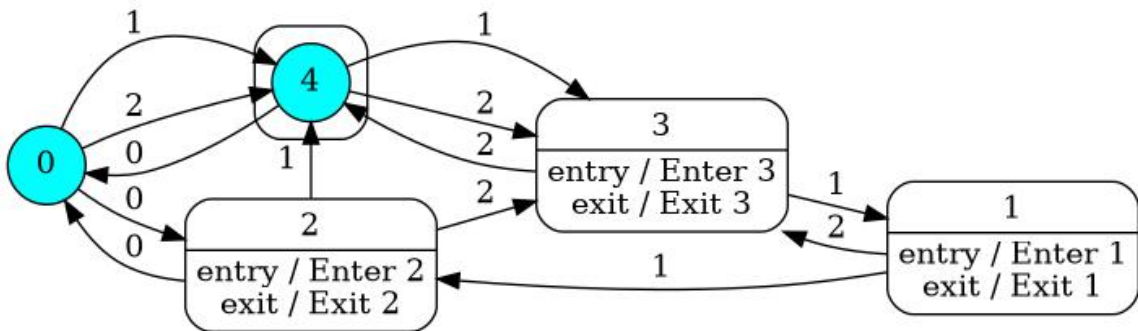
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (0);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (2);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (0);
        default: return (0); }
    default: return (0);
}
} // f()

```

Quale dei seguenti **UML** state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:





3.

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di consistenza" che è parte della "requirements validation activity".

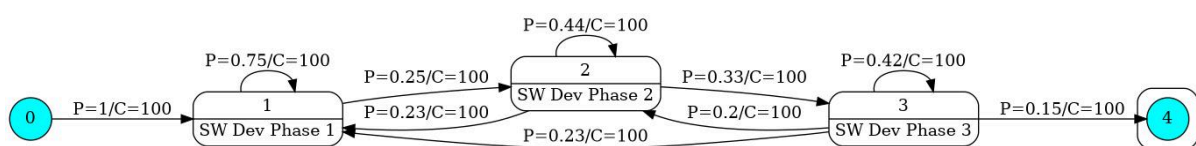
Scegli un'alternativa:

1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
2. Assicurarsi che per ogni requisito esista un insieme di test che lo possa verificare.
3. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di iterazioni della "SW Dev Phase 1"?

Scegli un'alternativa:

1. 0.773915
2. 3

3. 11.8193
